AGENTPULSE: MASTER SCIENTIFIC & TECHNICAL REPORT

A Lightweight, Self-Hostable Observability SDK for Continuous Grounding-Risk, Tool-Claim, and Drift Monitoring in Multi-Agent LLM Systems

Category: Industry-Trend AI Engineering	**Primary Benchmark:** Qwen/Qwen3-8B-GGUF:Q4_K_M	**Evaluation Cascade:** all-MiniLM-L6-v2 + DeBERTa-v3-small
Reasoning Strategies: Direct, CoT, AoT	**Label Agreement:** kappa = 0.922 (two LLM-judge passes, 50 cases)	**Tests (25 Oct 2026):** **422 backend + 136 dashboard passed**

1. Executive Summary & Problem Formulation

Modern enterprise AI systems are rapidly migrating from isolated single-prompt LLMs to collaborative multi-agent Directed Acyclic Graphs (DAGs) (e.g. LangGraph, AutoGen, CrewAI). In these pipelines, specialized autonomous agents decompose queries, retrieve evidence, execute deterministic calculations, verify assertions, and synthesize reports.

The Multi-Agent Observability Blindspot: Classical APM tools (Datadog, Prometheus, New Relic) and basic OpenTelemetry collectors assume that an **HTTP 200 payload with non-empty text signifies operational success**. In multi-agent DAGs, the most destructive failures produce zero software exceptions:

1. Ungrounded Hallucination Propagation: Upstream retriever/planner fabricates a citation or premise; downstream agents accept it as factual ground truth, compounding errors into executive reports.

2. Tool-Claim Fabrication: An agent queries a database returning 3 records, but claims in natural language: *'We verified 14 matching accounts.'* No API error occurs, but the assertion is ungrounded.

3. Inter-Agent Contradiction: Specialized agents operating over the same prompt generate mutually exclusive conclusions without reconciliation.

4. Silent Semantic & Behavioral Drift: Subtle prompt updates, model quantization, or retrieval corpus changes degrade reliability over time before explicit failures occur.

2. System Architecture & Core Product Capabilities

AgentPulse is a self-hosted observability prototype (version 0.1.0): a lightweight client SDK, a FastAPI/SQLite backend with a durable job queue, a separate worker that runs two small local models on every span, a regex tool-claim check, a drift detector, and a React dashboard. One worker evaluated 3.9 spans/s on a 16-thread development machine (experiments/results/throughput_benchmark.json).

• **Fail-Open Async SDK:** Non-blocking memory buffer with node wrapper overhead of **0.005 ms (P50)** and a synthetic in-memory enqueue loop of 5.4M spans/sec (not network throughput).

• **Two Local Models, Every Span:** all-MiniLM-L6-v2 (~13 ms) and nli-deberta-v3-small (~70 ms; latency profile of 2026-08-18) both run on every span, with no LLM judge and no API cost.

• **Tool-Claim Check:** regex extraction of stated counts vs the tool's recorded result, **0.07 ms** per call, F1 0.842 on 19 hand-written cases but **F1 0.000 on real agent traces** (TOOL_CLAIM_EXTERNAL_TEST_REPORT.md).

• **Drift:** DRIFT_DETECTED when the mean embedding of an agent's latest 12 outputs is more than 0.30 (cosine) from the mean of its first 20; an Agent Stability Index (0-100) is kept beside it.

3. Mathematical Formulation & Latency Profiling

Composite Grounding Risk Score Formulation: For span *s* with input context C_{in}, output O_{out}, and tool records *T*:

$$R(s) = [0.40 * G(C_{in}, O_{out}) + 0.25 * R_{tool}(T, O_{out})] / (\text{sum of the weights of the signals present}),\quad G = P_{contradiction} + 0.3 * P_{neutral}$$

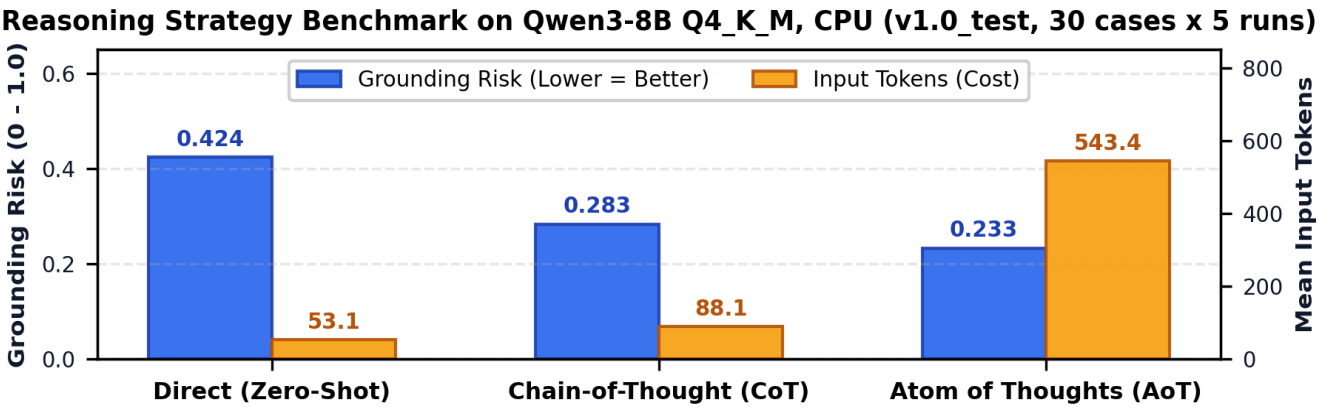
The tool term counts only when a claim was extracted. High risk above 0.7, medium above 0.4. The disagreement score (weight 0.20 declared) and the similarity (0.15) are stored but not summed (backend/app/services/evaluator.py _aggregate_risk). Shipped grounding score on the 30-case test split: F1 0.963, FPR 0.059 at both cut-offs; on the 47 truly held-out cases F1 0.927 (0.7) / 0.952 (0.4), FPR 0.074 (GROUNDING_RECALIBRATION_2026-10.md T3c, T3d). 5 of the 30 test cases copy dev cases; the 0.3 weight is a design choice.

13-Layer Latency Profile Breakdown (25 Repeated Runs, 16-Core CPU, run of 2026-08-18; layers 8, 9 and 13 are not measurements):

Layer Index & Description	Mean	P50	P95	Std Dev	Measurement Scope
1. Prompt Preparation	0.001 ms	0.001 ms	0.001 ms	0.000 ms	Template string formatting
2. Model Inference Dispatch	0.011 ms	0.008 ms	0.019 ms	0.010 ms	Canned-text adapter call, model weights not loaded
3. Token Generation Telemetry	0.003 ms	0.003 ms	0.006 ms	0.002 ms	Same call, as the adapter reports it (a time, not tok/s)
4. Agent Node Execution	**0.001 ms**	**0.001 ms**	**0.001 ms**	0.000 ms	**Stand-in (builds a dict); SDK wrapper not called**
5. Local Tool Execution	0.003 ms	0.003 ms	0.006 ms	0.001 ms	Stand-in (builds a list of 10 dicts); no tool run
6. Local Vector Retrieval	15.36 ms	10.68 ms	12.19 ms	24.29 ms	SentenceTransformer top-k search
7. SDK In-Memory Enqueue	0.025 ms	0.016 ms	0.032 ms	0.037 ms	Thread-safe deque append
8. HTTP Ingestion Overhead	n/m	n/m	n/m	n/m	Random number in the script; no HTTP request sent
9. Evaluation Worker Dispatch	n/m	n/m	n/m	n/m	Random number in the script; nothing dispatched
10. MiniLM Embedding Inference	**13.33 ms**	**12.81 ms**	**14.95 ms**	1.483 ms	**Embedding similarity, every span**
11. DeBERTa NLI Cross-Encoder	**70.82 ms**	**66.92 ms**	**71.88 ms**	21.46 ms	**NLI classification, every span**
12. Full Evaluation Cascade	**90.81 ms**	**90.96 ms**	**97.79 ms**	4.339 ms	**MiniLM + DeBERTa + tool check**
13. Entire Multi-Agent DAG	**n/m**	**n/m**	**n/m**	n/m	Computed from layers 2, 6 and 12, not timed

4. Reasoning Strategy Evaluation: Direct vs. CoT vs. AoT

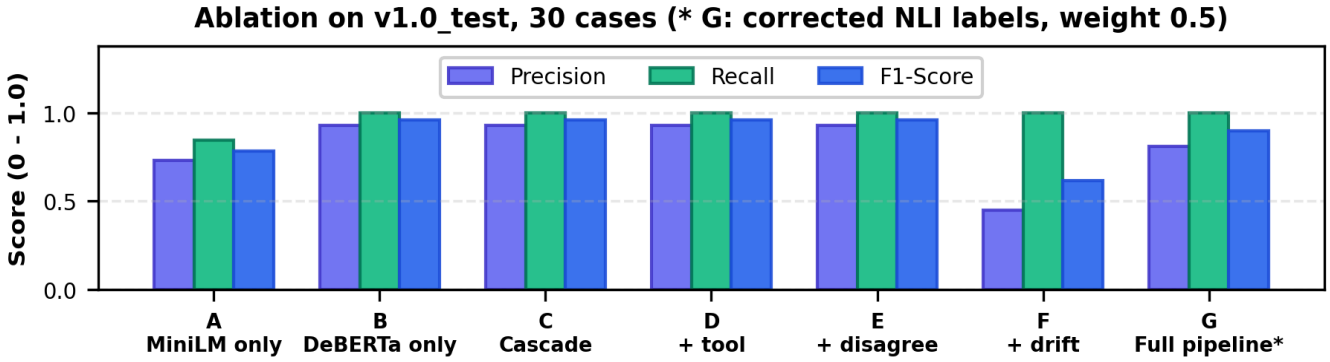
Evaluated on **Qwen/Qwen3-8B-GGUF:Q4_K_M** (llama.cpp, CPU only) across v1.0_test (30 cases x 5 runs per strategy, run of 2026-08-23, experiments/results/reasoning_strategy_results.json). Latency is the mean per strategy execution; risk differences between strategies are inconclusive:



Reasoning Strategy	Latency (ms)	Tokens In	Tokens Out	Mean Risk	Contradiction Rate	Empirical Conclusion
DIRECT (Zero-Shot)	11,564 ms	53.1	37.5	0.424	0.133	Fastest and cheapest; highest mean risk, but within noise.
COT (Chain-of-Thought)	45,423 ms	88.1	186.4	0.283	0.127	Middle on latency and tokens; no risk advantage shown.
AOT (Atom of Thoughts)	85,215 ms	543.4	319.7	0.233	0.000	Slowest; ~8.5x DIRECT output tokens; zero contradictions in 150 runs.

5. Architectural Ablation

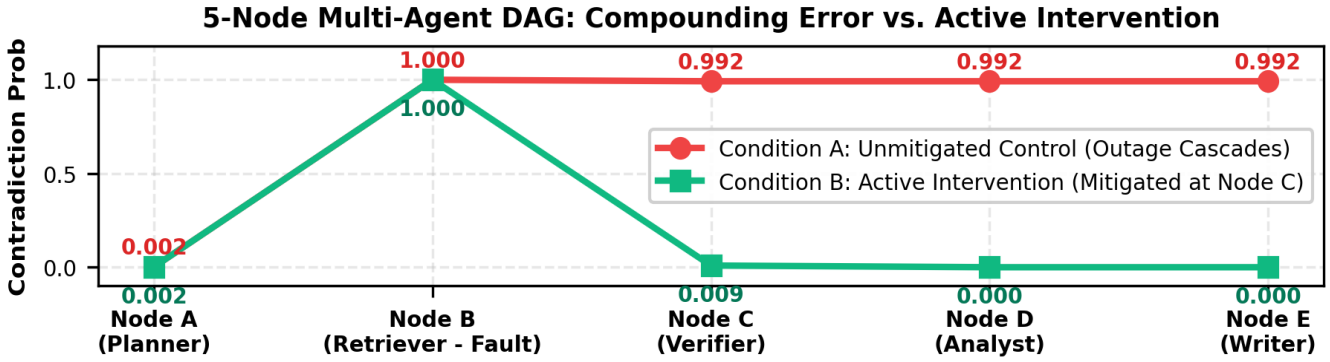
Seven configurations on v1.0_test (30 cases; thresholds chosen on the 21-case dev split and applied unchanged), run of 2026-09-18 on the PyTorch path (experiments/results/ablation_results.json). Configuration G is shown re-scored on corrected NLI labels at the 0.5 weight it ran with (GROUNDING_RECALIBRATION_2026-10.md T8); it has not been re-run at the shipped 0.3. An earlier comparison against four baselines (2026-08-18, a 20-case test file) predates the grounding-score fixes and was not re-run, so it is no longer shown.



System / Configuration	Precision	Recall	F1-Score	FPR	FNR	Latency	Key Trade-off
A: MiniLM embedding only	0.733	0.846	0.786	0.235	0.154	19.10 ms	Similarity alone misses contradictions.
B: DeBERTa NLI only	0.929	1.000	0.963	0.059	0.000	132.09 ms	The signal that does the work.
C: MiniLM + DeBERTa (shipped)	**0.929**	**1.000**	**0.963**	0.059	0.000	**151.19 ms**	Both models on every span; MiniLM adds no F1.
D: NLI + tool-claim	0.929	1.000	0.963	0.059	0.000	132.13 ms	Single-agent cases cannot exercise it.
E: NLI + disagreement	0.929	1.000	0.963	0.059	0.000	281.02 ms	Same; 0 of 10 on real multi-agent traces.
F: NLI + drift	0.448	1.000	0.619	0.941	0.000	145.58 ms	Cold-start drift on non-temporal data.
G: Full pipeline*	0.812	1.000	0.897	0.176	0.000	165.18 ms	*Corrected labels, weight 0.5; file says 0.963.

6. 5-Node Compounding Error: Control vs. Active Intervention

Evaluated on a 5-node pipeline (Planner -> Retriever -> Verifier -> Analyst -> Writer) with an ungrounded assertion injected at Node B. Risks are the corrected-label re-run at weight 0.5 (experiments/results/grounding_recalibration_2026-10-22_compounding.json); the contradiction probabilities in the chart do not depend on the label order. At the shipped weight 0.3 the intervening nodes C-E score about 0.30 (low), Node A 0.006 (derived from the stored probabilities, not re-run):



Pipeline Node	Condition A: Control (Risk / Contra Prob)	Condition B: Active Intervention (Risk / Contra Prob)	Operational Outcome
Node A (Planner)	0.008 / 0.002	0.008 / 0.002	Premise compared with itself: entailment 0.9865.
Node B (Retriever - Fault)	**1.000 / 1.000 (Fault Injected)**	**1.000 / 1.000 (Fault Injected)**	Fabricated citation injected into stream.
Node C (Verifier)	0.996 / 0.992 (Propagated)	**0.504 / 0.009 (Contradiction stopped)**	**Contradiction no longer propagates; output is neutral, not entailed.**
Node D (Analyst)	0.996 / 0.992 (Corrupted)	**0.500 / 0.000 (Neutral)**	Medium at weight 0.5; about 0.30 (low) at 0.3.
Node E (Writer)	0.996 / 0.992 (Corrupted)	**0.500 / 0.000 (Neutral)**	Medium at weight 0.5; about 0.30 (low) at 0.3.

7. Graded Drift Benchmark with Negative Controls

11 synthetic scenarios with graded shifts and negative controls (experiments/drift_scenarios.py, the per-output distance used in August). 2 of 5 anomalies were detected and 0 of 6 non-anomalies raised an alert, but no scenario reached the 0.30 centroid threshold, so that path was untested (DRIFT_EXPERIMENT_REPORT.md). The shipped rule now compares window means (first 20 vs latest 12 outputs): offline on a held-out real-trace split, measured once, 1.5% false alarms and 91.9% detection, with only 24.5% of sessions long enough to report (DRIFT_REAL_TEXT_DIAGNOSIS_REPORT.md 11.3):

Scenario / Condition	Category	Magnitude	True Anomaly?	Detected?	False Alert?	Detection Delay	Final ASI
Prompt Formatting Change	prompt_drift	0.10	No	No	No	N/A	100.0 / 100
Prompt Tone Shift	prompt_drift	0.25	No	No	No	N/A	99.7 / 100
Prompt Template Rewrite	prompt_drift	0.50	Yes	No	No	N/A	98.5 / 100
Model Version Update	model_drift	0.50	Yes	No	No	N/A	98.5 / 100
Temperature Shift (0.1 to 0.9)	hyperparam	0.35	Yes	No	No	N/A	99.4 / 100
Tool Frequency Fluctuation	tool_entropy	0.25	No	No	No	N/A	99.7 / 100
Uncalibrated External Tool	tool_entropy	0.60	Yes	**Yes**	No	**1 span**	**82.7 / 100**
Hallucination Burst	quality_regress	0.75	Yes	**Yes**	No	**1 span**	**96.5 / 100**
Negative Control: Paraphrasing	neg_control	0.12	No	No	No	N/A	100.0 / 100
Negative Control: Valid Tool	neg_control	0.15	No	No	No	N/A	99.9 / 100
Negative Control: Invariant Flow	neg_control	0.00	No	No	No	N/A	100.0 / 100

8. Label Agreement & 9. Test Suite

Label agreement (two LLM-as-judge passes, not human annotators):
- **Dual-evaluated cases:** v1.0_dev 15, v1.0_val 15, v1.0_test 20 (full splits 21 / 22 / 30; 5 test cases copy dev cases).
- **Observed Agreement (p_o):** 0.960 | **Chance Agreement (p_e):** 0.490
- **Cohen's Kappa: 0.922** between two LLM passes; not human-verified ground truth (LABEL_AGREEMENT_REPORT.md).

Automated tests, run on 25 Oct 2026 (main at 2fefb91):
- **Backend:** **422 passed** (pytest tests/, 335.91 s, real models loaded)
- **Dashboard:** **136 passed** in 13 files (npx vitest run)
- **Coverage:** SDK queue and decorators, ingest, durable job queue and crash recovery, NLI grounding, tool-claim, drift, alerting, migrations, retention, dashboard adapters.

10. Examiner Hostile Review & Viva Defense Summary

Q1: What exactly did your latency timers measure?
Defense: We profiled 13 layers (experiments/latency_profiler.py); layers 8, 9 and 13 are not measurements (two random numbers and a sum, marked n/m) and layers 4 and 5 time small stand-ins. String templating took <0.002 ms, the SDK decorator added 0.005 ms (P50, 2,000 calls, benchmarks/run_benchmarks.py), local MiniLM forward pass took 12.8 ms (P50), and DeBERTa NLI took 66.9 ms (P50). We distinguish between in-memory queue append (0.016 ms) and neural model inference.

Q2: Do the extra signals beat NLI alone?
Answer: No, not on this data. On the 30-case test split, tool-claim (D) and disagreement (E) tie with NLI alone (B) because single-agent cases cannot exercise them, drift folded into the score (F) is worse (F1 0.619), and the full pipeline (G) is 0.897 on corrected labels. On real traces tool-claim scored F1 0.000 and disagreement found 0 of 10 labelled contradictions; its alert is withdrawn. Grounding and drift are the signals that held up.

Q3: Does Atom of Thoughts (AoT) actually improve factual grounding?
Defense: Not shown. On Qwen3-8B Q4_K_M (30 cases x 5 runs), AoT produced about 8.5x DIRECT's output tokens (319.7 vs 37.5) and took 85.2 s per execution against 11.6 s. It had the lowest mean grounding risk (0.233, against 0.283 for CoT and 0.424 for DIRECT) and no contradictions in 150 runs, but the spread between strategy means (0.191) is smaller than the within-strategy standard deviation (0.377), so the risk ordering is inconclusive. The zero-contradiction result is the part the data supports.

Q4: How does AgentPulse prevent false alerts on benign phrasing shifts?
Answer: In the synthetic scenarios the three negative controls measured centroid distances of 0.000 to 0.002 and raised 0 alerts; the 0.30 threshold was not tuned, and no scenario reached it, so 3 of 5 real anomalies were missed. On real agent text the shipped window-mean rule gave 1.5% false alarms on a held-out split, but only on the 24.5% of sessions long enough to report (DRIFT_REAL_TEXT_DIAGNOSIS_REPORT.md).